DdoS Attack and Firewall Mitigation

Name: Aathityan 😈

Degree in IT: Comp. Forensic (Sem 2)

Difficulty: Easy

COMPANY : BANKING WEB APP (SecureBank Online)

Before we start, we Need to know what is DdoS attack. DdoS, or Dos attack is also known as distributed denial of service. It is a cyberattack that overwhelmes a server or a website, with a flood of internet traffic, compromised from (botnet). In HUMAN words, its like forcing 10s of thousands of humans to enter a small, classroom sized door, resulting uncontrollable chaos.

DO NOT TEST DdoS/ DoS attack in a public server/web.

DdoS attack in a public server will result in :

- **RM100.K / 24,000 USD FINE**

OR

- **7+ YEARS IN JAIL**

OR

- **BOTH**

THIS IS HIGHLY FOR EDUCATIONAL PURPOSE ONLY

OBJECTIVES INCLUDES:

- Creating a vulnerable banking web app for testing.
- Simulate a DdoS attack.
- Observing the side effects, from user experience, and in wireshark
- Implementing a mitigation steps, which is a firewall, to block suspicious or sudden floods of attack.
- Documenting the results, and provide a conclusion.

CyberSec Tools used:

TOOLS	PURPOSES
PYTHON3 HTTP SERVER	TO CREATE A VULN. WEB APP
APACHE BENCH (ab)	TO SIMULATE DDOS ATTACK
WIRESHARK	TO MONITOR TRAFFIC AND PACKETS
nftables	TO CREATE FIREWALL AND LIMITING VISIT RATES
PYTHON, HTML	TO CREATE FRONT END OF WEB APP

FUN PART : GET STARTED !!!

STEP 1 : (CREATING THE WEB APP.)

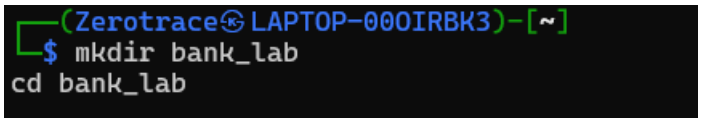
I created a banking webiste, just to show how bad this DdoS Attack is in a public any server, or a public website.

IN KALI LINUX:

Create a project folder:

```
mkdir ~/weakbank

cd ~/weakbank
```



Then, Create the Vuln. Server file:

```
nano server.py
```



Then, paste your web app code. It can be any web app, such as banking web, shopping web, and so on.
Then click

```
CTRL + O

ENTER

CTRL + X
```

→ TO SAVE THE CODE

After that, enter

Python3 server.py → TO RUN YOUR CODE.

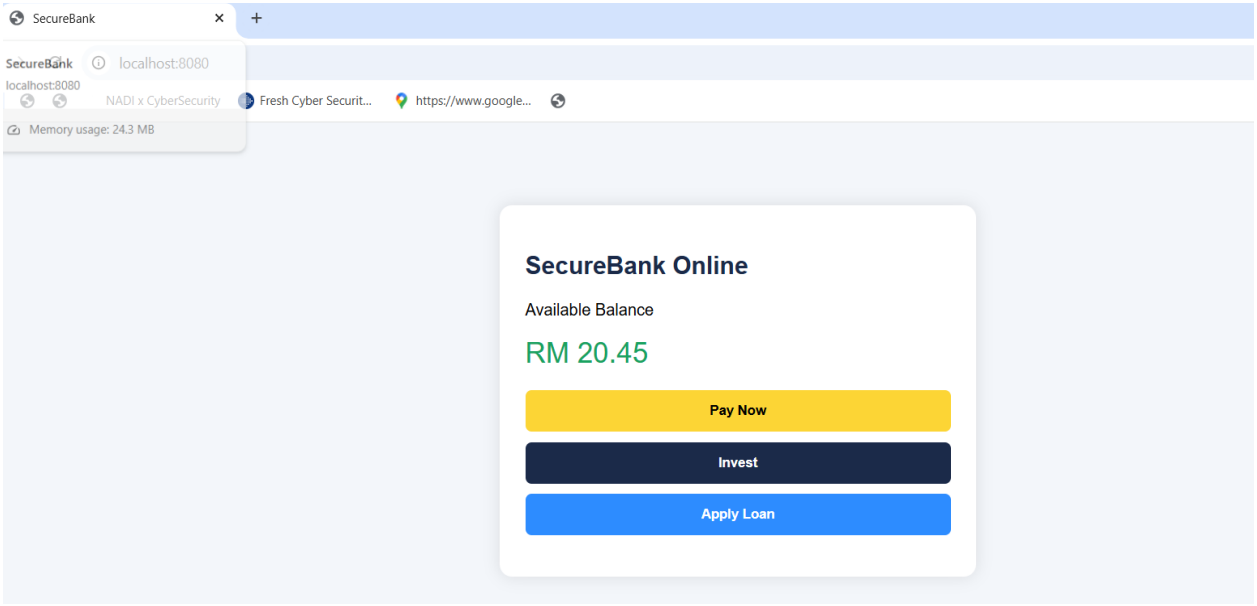
```
┌──(Zerotrace㉿LAPTOP-000IRBK3)-[~/weakbank/weakbank]
└─$ python3 server.py
Bank server running on port 8080
```

Then, you should see

http://localhost:8080

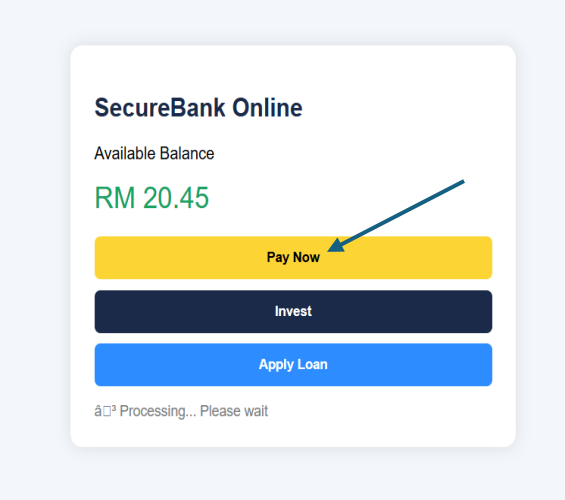
enter in your browser to check your web app.

In my case, since I code my web as a banking web app, when I click on the link, it should look like this:

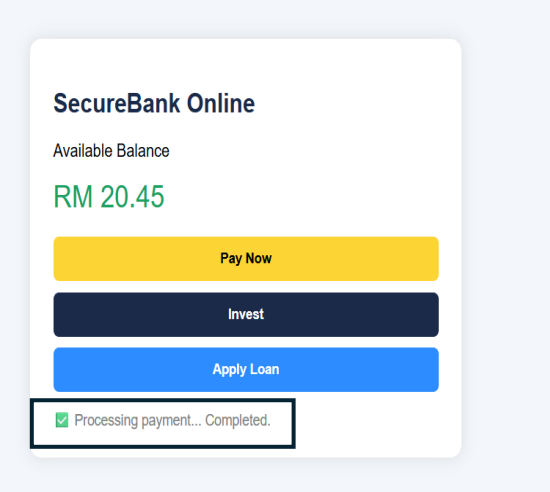


In my web app, there are a bunch of buttons to click, such as “Pay Now” , Invest and Apply Loan. To check if my web app actually works fine, I click on one of those buttons, and wait for the response.

1. I clicked on “ Pay Now”



2. It responded with, “ Process Payment Complete” , it means my web works fine.

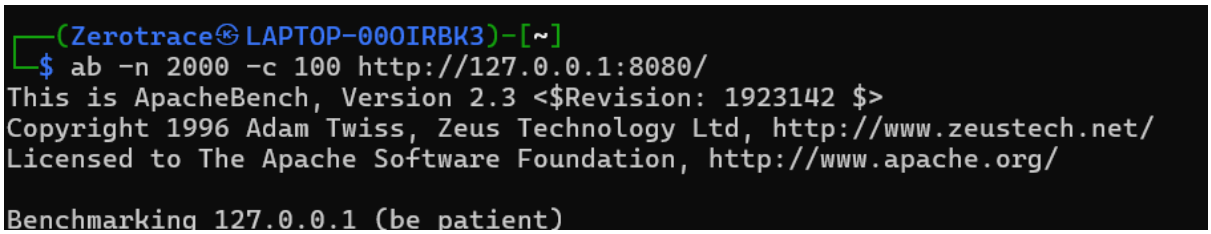


STEP 2: (ATTACK)

This is where it gets fun!!!

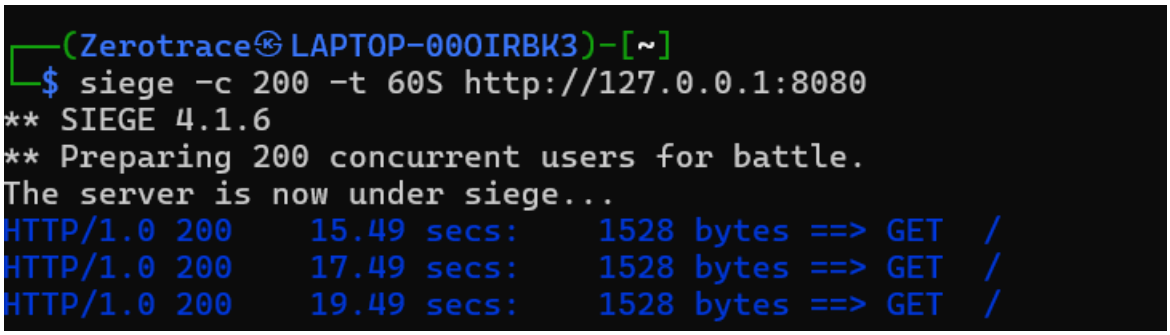
When your ready, open new KALI LINUX TERMINAL, and use the command :

```
ab -n 3000 -c 200 http://127.0.0.1:8080/
```



Or

```
siege -c 50 -t 30S http://127.0.0.1:8080
```



For the command:

- ab -n 3000 -c 200 http://127.0.0.1:8080/
  what it does is that it sends 200 users (-c 200), all at the same time until for 3000's of times (-n 3000).

- siege -c 50 -t 30S http://127.0.0.1:8080
  siege is closer to a real world ddos attack.
  Users continuously hitting the site, and it also overlaps requests.
  This way, the server will never gets time to recover.

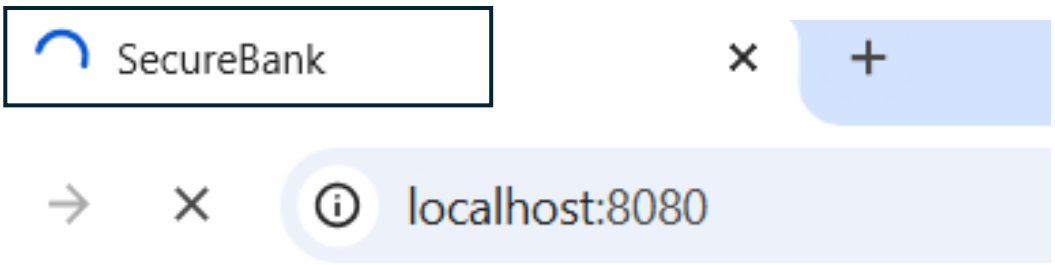
STEP 3 (OBSERVE THE ATTACK)

IN USER EXPERIENCE:

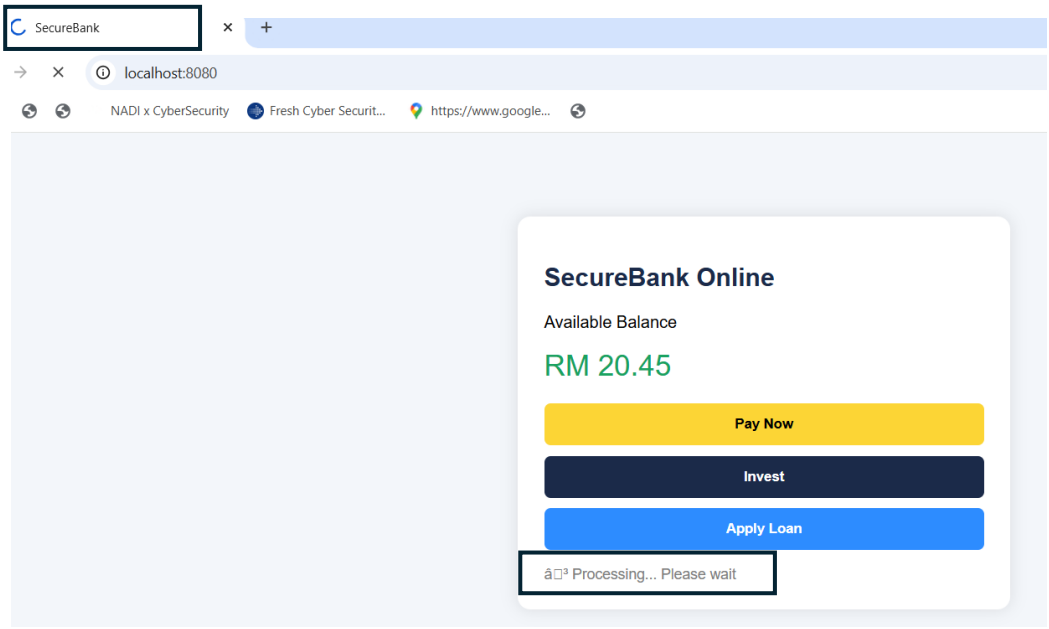
Such as a banker, or a customer / client

What will they observe is :

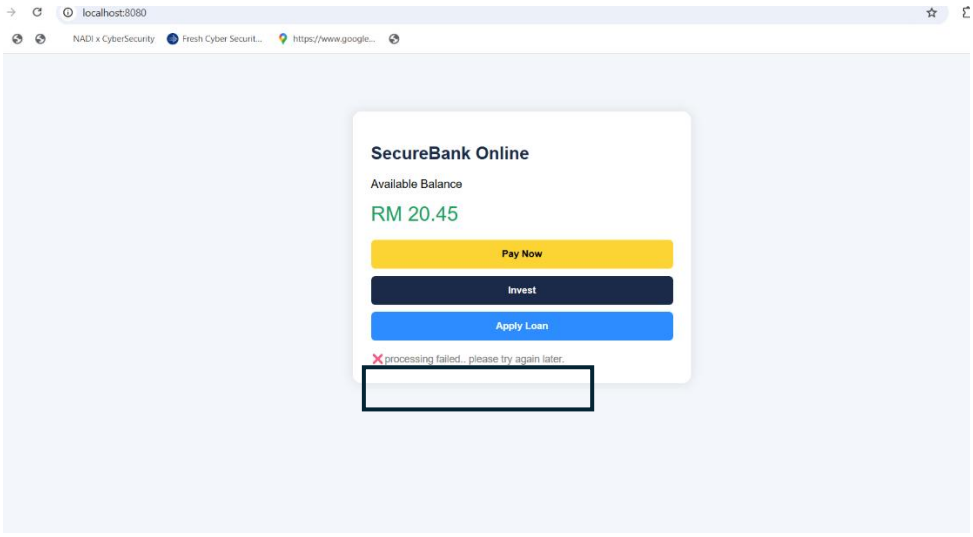
1. The web app keep on loads



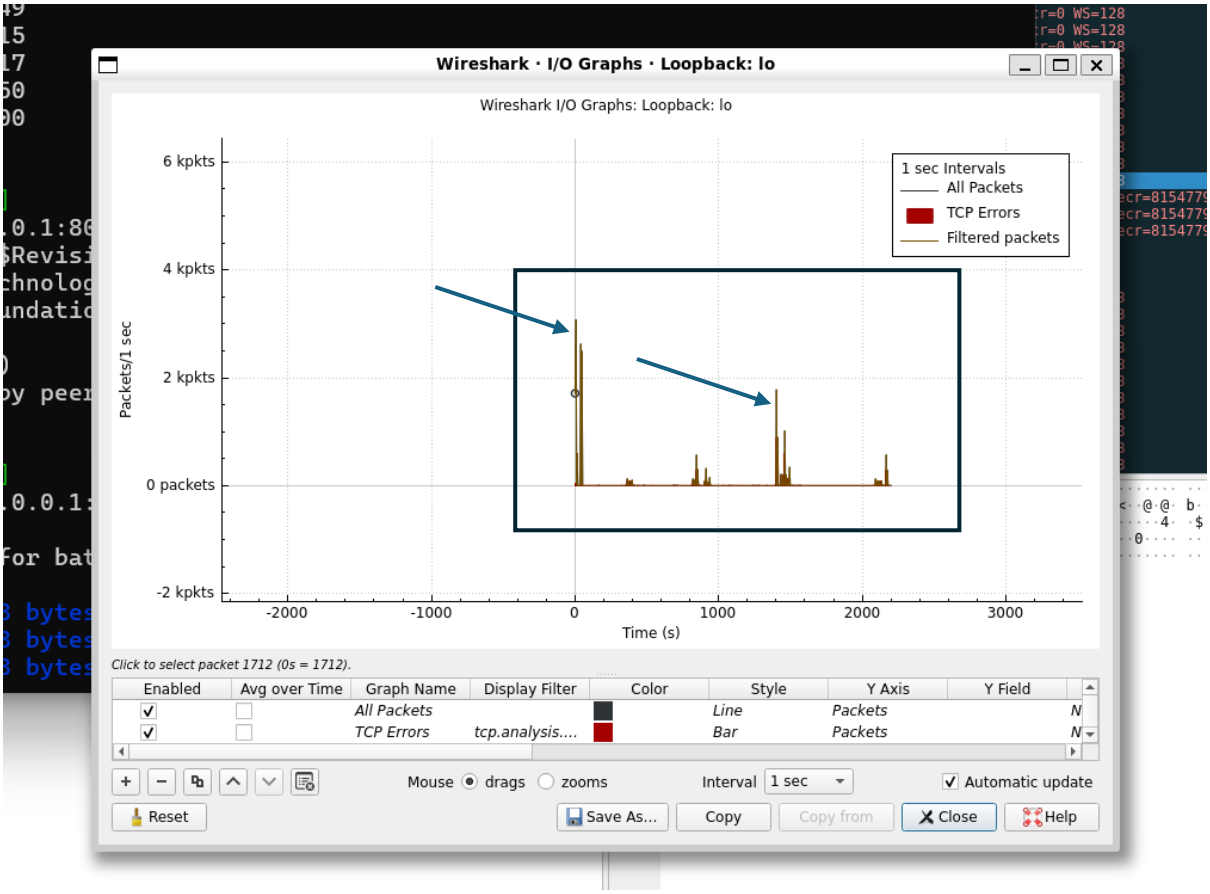
2. When the user clicked a button, in this case, " Pay Now" , the website will stuck in processing payment.



3. Which will result as “Payment Failed” , as the web could not handle multiple requests at once.



IN SOC (SECURITY OPERATION CENTER) POV:



In wireshark, click statistic and I/O graph to check the traffic activity.

Imagine, there is:

- 50 web visitors per hour, during weekdays
- 100 web visitors per hour, during weekends

To check if there is a sudden weird activity, such as DdoS attack, the smart way to check is by looking at the I/O graph above.
Since I created a fake Banking web where there is no visitors, the sudden spike here is DdoS attack.
In the graph, you can observe that, there is quiet a lot of visitors, in really short amount of time, resulting late process, or even failing to process, as the web couldn't handle a lot of visitors at once, in a short amount of time.

WORST CASE SCENARIO:

Imagine your trying to pay for your groceries, and you didn't bring your cash.
So the only way for you to pay is by QR Payment or Bank transfer.
When you are trying to make a payment using a Banking web app, then it kept on reloads over and over again.
You will then refuse to pay for your groceries and go back.
Now imagine, since everyone is using the Banking web app at the same time and couldn't pay.
It will result on big loss for companies and businesses, and could also effect the whole country's economy in a single day.

STEP 4 (MITIGATION / STEPS TO AVOID THE ATTACK)

FIREWALL !!

In order to prevent from DdoS Attack, SOC teams analyse the website's traffic for the first few months. That doesn't mean that the Firewall is still not set up, it is still on monitor mode, where it will alert the SOC teams about a bad requests, and also to check the IP address (e.g : 1.2.3.4) given from the Firewall.

Once the SOC teams have analysed the rate of visitors visiting the website, during the weekdays, weekends, and certain holidays, they will setup a limit using the Firewall. So that if it detects high traffic activity during weekdays, which is quiet unusual, it will block the certain IP address.

Keep in Mind, there are also 2 types of attack, DoS and DdoS.

Firewall is good at blocking DoS, as it is only 1 attacker, using 1 IP address.

However

Firewall is quiet bad when it comes to blocking DdoS. Since DdoS is 1 attacker, using multiple devices (IP adresses), sending requests.

Many companies uses AWS Shield, and more to block the DoS attacks.

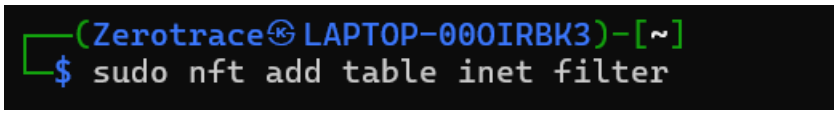
After I observed the effects of DdoS attack in my Banking web app, I came up with a solution, which is to install a Firewall.

In order to install a firewall and set limitations based on the visiting rate:

First, use the command :

```
sudo nft add table inet filter

sudo nft add chain inet filter input { type filter hook input priority 0 \; policy drop \; }
```



- Nft' is the command line's tool for nftables
- add table inet filter' = creating a new table called filter in the inet family.

Table is like a containers for the firewall's rules. Inet family handles both ipv4, and ipv6 traffics. Its like creating a workspace for the firewall.

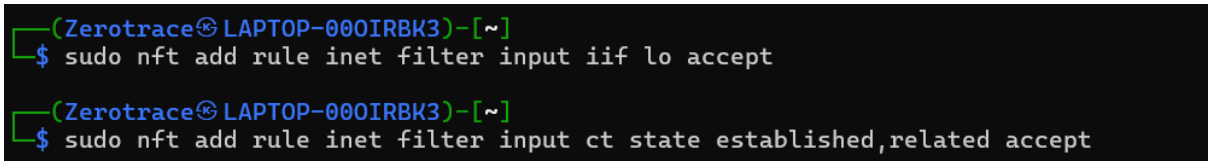


- add chain inet filter input'creates a chain, called input, inside the filter table.
- type filter' will filter traffic
- hook input' is the rules applied, to incoming traffics
- priority 0' order in which rules are applied.
- Policy drop drop any packet that doesn't match a rule.

Last command :

```
sudo nft add rule inet filter input iif lo accept

sudo nft add rule inet filter input ct state established,related accept
```



STEP 5 (THE RESULTS AFTER INSTALLING FIREWALL)

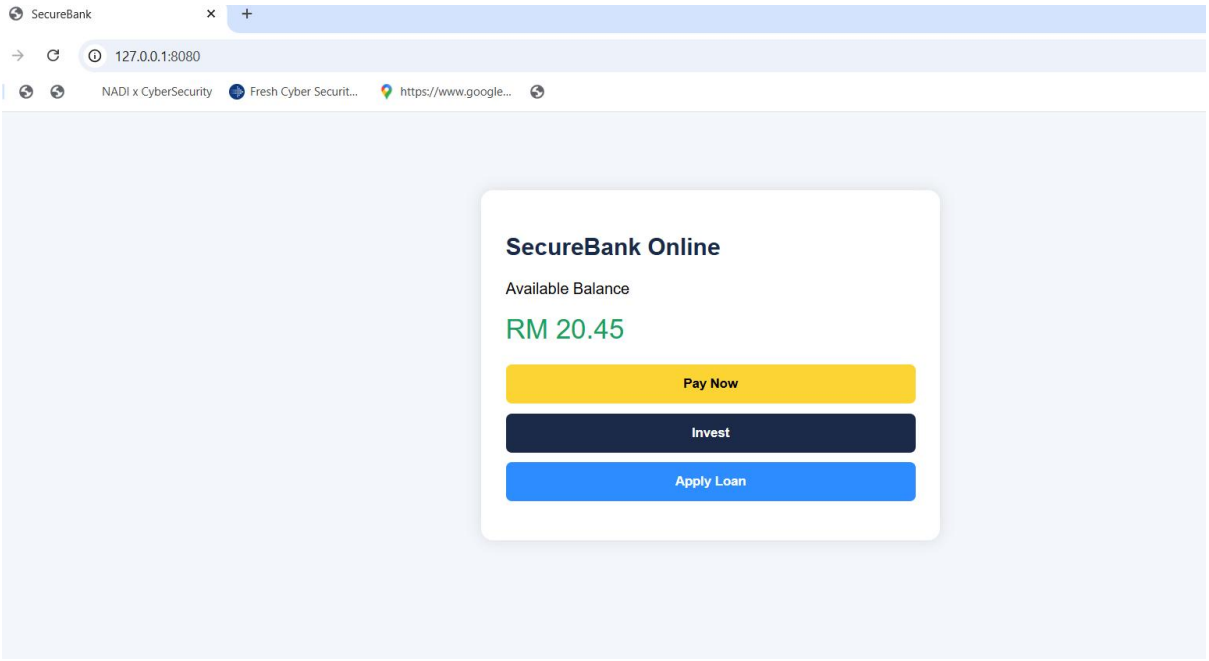
Now try DoS Attack again ..

ab -n 3000 -c 200 http://127.0.0.1:8080/

```
┌──(Zerotrace㉿LAPTOP-00OIRBK3)-[~]
└─$ ab -n 2000 -c 100 http://127.0.0.1:8080/
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
```

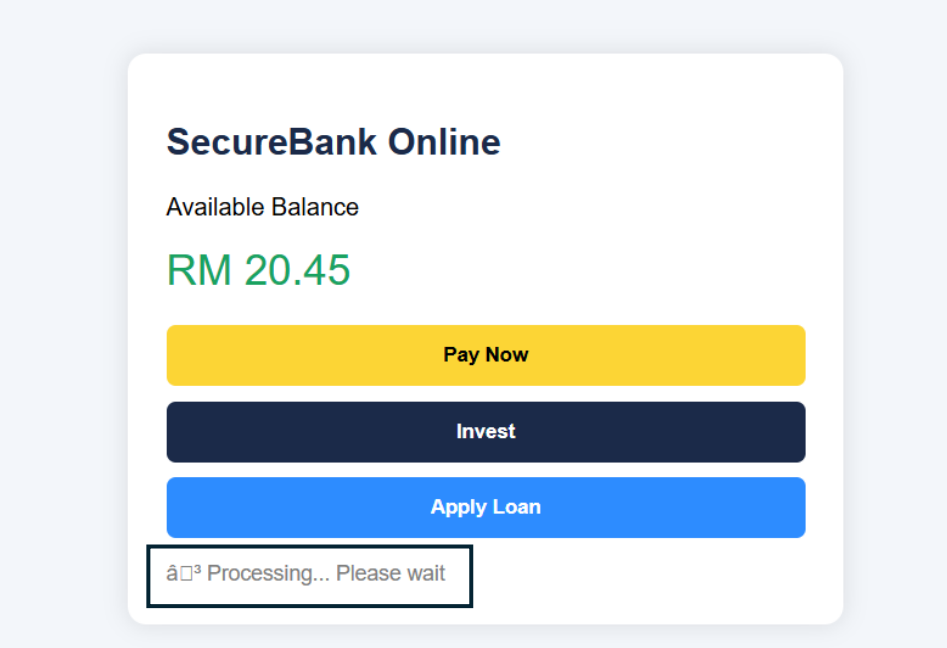
USER EXPERIENCE POV:

Then, enter the website you have created, and reload it (CTRL + R)

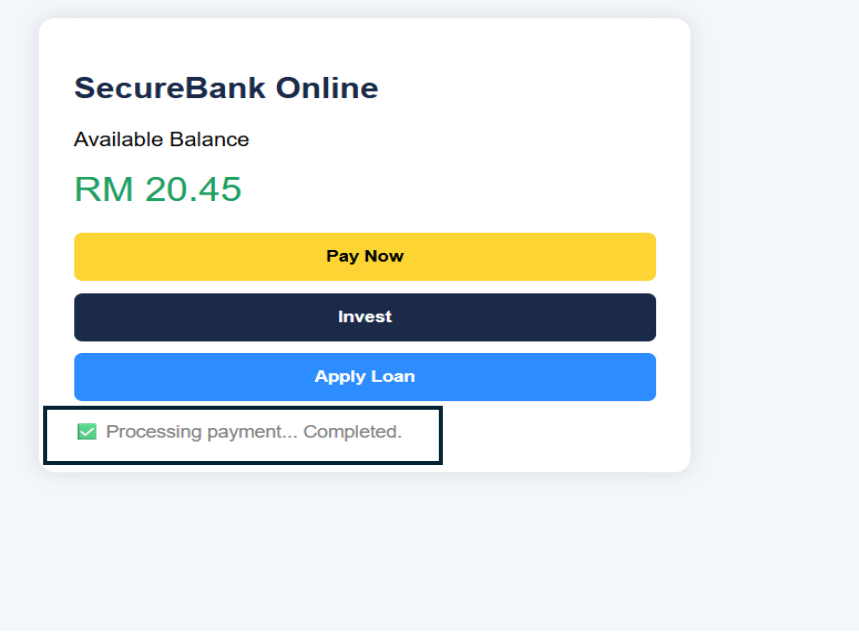


Here, once I clicked CTRL + R, the reload timing just takes seconds, even during an attack.

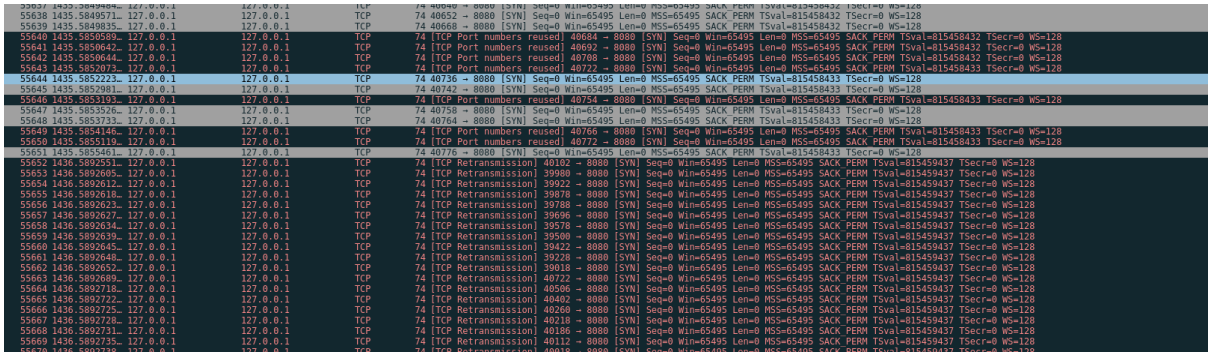
To double check, I clicked “Pay Now”.



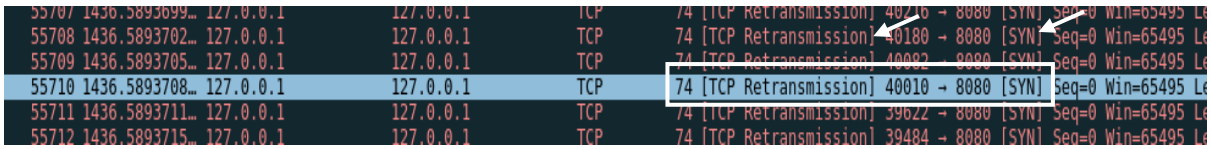
In seconds, it returns with “ processing successful”



SOC POV:



In wireshark, when an IP is being blocked by a Firewall, SOC sees a black block of requests being blocked. In order to find out that it is blocked by Firewall, SOC teams looks for [SYN] , and [TCP RETRANSMISSION].



CONCLUSION 🤓

To show the effects of a DDoS attack, I had developed a vulnerable Financial Banking web app to demonstrate the attack. The web application had faced delayed responses and simulated service unavailability using load testing tools like Apachebench and Siege, showing the dangers of excessive traffic.

After that, firewall based mitigation was put into place using nftables with rules to reject more requests while permitting valid traffic. Normal user requests are successfully processed after the firewall was installed, but attack traffic was either blocked or subject to a rate limit. The mitigation was verified by Wireshark captures, which revealed dropped packets and retransmissions for the excess traffic.

This exercise shows that appropriate firewall configuration guarantees service availability, restricts resource exhaustion, and lessens the impact of malicious traffic even in the case of a simulated DDoS attack. The lab shows how important a network security measures are to safeguarding online services.